

Android stale-cache diagnosis

Inflammation Tracker PWA

Notes for follow-up — 2026-06-11

Working notes covering why an Android device may continue serving the previous version of `index.html`, `app.js`, or `sync.js` after a code change is deployed. Captured for later because the symptom recurred on 2026-06-11 after the `naproxen` column was added.

Contents

1	Symptom	1
2	Three suspects	1
2.1	Suspect 1 — HTTP cache underneath the service worker	1
2.2	Suspect 2 — the service worker script itself is cached	2
2.3	Suspect 3 — server-side <code>Cache-Control</code> headers	2
3	Recommended order of work	3
4	Reference: where the relevant code lives	3

1 Symptom

After deploying updated app code (e.g. a new form field, a CSS tweak, a logic fix), the Android phone continues to render the previous version even after force-closing the PWA and re-opening it. Desktop browsers pick up the change on next reload; Android does not.

This is *not* the same as the v11-era “`.htaccess` basic-auth stranded SW” trap — the service worker itself is already mitigated for that case (tolerant install, network-first fetch, non-OK never poisons the cache). The current symptoms point at caching layers *above* the service worker, on the device.

2 Three suspects

2.1 Suspect 1 — HTTP cache underneath the service worker

The service worker fetch handler in `app/sw.js` does:

```
const res = await fetch(req);
```

No `cache` option is supplied. The browser’s HTTP cache sits *below* the service worker: when the SW does `fetch(req)`, that request is allowed to be satisfied from the disk cache before ever touching the network. So “network-first” from the SW’s point of view can in practice be “HTTP-cache-first”. Android Chrome is more aggressive about this than desktop browsers, especially for assets with stable URLs and no cache-busting query string (`app.js`, `sync.js`, `index.html` all have stable URLs in this repo).

How to check: Attach the Android phone to a Mac with a USB cable and open `chrome://inspect` in desktop Chrome. With the PWA open on the phone, click *inspect* to attach DevTools. Then:

- DevTools → Network: reload the page and look at the *Size* column for `app.js`, `sync.js`, `index.html`. If the column shows (disk cache) or (memory cache), the SW handed over a cached copy without re-validating with the server.
- DevTools → Application → Cache Storage: expand `infram-shell-vXX`. If `vXX` is older than the latest `CACHE_VERSION` in `app/sw.js`, the new SW has not yet activated — jump to Suspect 2.
- DevTools → Application → Service Workers: ensure status is *activated and is running*. If a worker is listed as *waiting*, the new SW is installed but the old one still controls the page; close all tabs/instances of the PWA, then re-open.

Proposed fix (one line): change the fetch handler in `app/sw.js` to bypass the HTTP cache:

```
const res = await fetch(req, { cache: "no-store" });
```

This forces every shell-asset fetch to go to the network, not the HTTP cache. The SW’s own cache (the **Cache Storage** entry it maintains as an offline fallback) is unaffected and still wins when the network is unreachable or returns a non-OK status. The trade-off is one extra round-trip per asset per load — negligible for seven small files.

2.2 Suspect 2 — the service worker script itself is cached

The service worker is registered in `app/app.js`:

```
navigator.serviceWorker.register("sw.js");
```

The default value of `updateViaCache` is `"imports"`, which means the browser is allowed to fetch `sw.js` *itself* through the HTTP cache. If the device’s HTTP cache is holding a stale copy of `sw.js`, the browser never sees the new `CACHE_VERSION` value, so the new SW never installs, never activates, and never wipes the old cache.

How to check: Same DevTools session as above, Application → Service Workers. The *Source* link next to the current SW will show its contents. If `CACHE_VERSION` in that source is older than the value currently in the repo, the SW script itself is stale.

Proposed fix (one line):

```
navigator.serviceWorker.register("sw.js", { updateViaCache: "none" });
```

With `updateViaCache: "none"`, the browser always bypasses the HTTP cache when checking for an updated `sw.js`. This is the canonical fix for “my new service worker is never activating”.

2.3 Suspect 3 — server-side Cache-Control headers

If the `.htaccess` file on the Hostinger deployment is setting a long `max-age` for `*.js` or `*.html` (or an `ExpiresByType` directive with a long default), then the HTTP cache is being *told* by the server to hold these files for hours or days. This is the upstream cause of Suspect 1: even with `cache: "no-store"` in the SW, if a long-lived response is already in cache from before the fix, it will not be re-fetched until the cache entry naturally expires.

How to check: Two options.

- On the server, view `.htaccess` and grep for `Cache-Control`, `ExpiresActive`, `ExpiresByType`, or `Header set Cache-Control`.
- From any browser, open DevTools → Network, reload the page, click `app.js`, look at the *Response Headers* pane. Note the `Cache-Control` and `Expires` values that the server is sending.

Proposed fix (server side — only if Suspect 3 turns out to be present): add the following to the `.htaccess`, scoped to the app's shell files:

```
<FilesMatch "\.(html|js|css|json)$">
  Header set Cache-Control "no-cache, must-revalidate"
</FilesMatch>
```

`no-cache` (despite the name) does not forbid caching — it forbids *using* a cached copy without revalidating with the server first. Combined with the SW's `cache: "no-store"` fetch, this guarantees the device sees the latest deployed code on every page load.

3 Recommended order of work

1. Apply Suspect 1 and Suspect 2 fixes together — both are one-line client-side changes in this repo, low risk, and address the most common Android-stale-PWA failure modes. Bump `CACHE_VERSION` once after the change so existing devices flush their old SW-managed cache on the next visit.
2. Re-test on the Android phone using the DevTools inspection steps above. In particular, watch for `app.js` loading with a non-cache *Size* value, and `CACHE_VERSION` in the running SW matching the repo.
3. Only if the symptom persists, investigate Suspect 3 via the `.htaccess` on Hostinger.

Important

Be wary of one false positive when testing: if a PWA is installed to the home screen and was last opened with the old code, the splash transition can briefly flash the old UI from a screenshot before the new code paints. Wait a full second after the splash before judging whether the new code is live.

4 Reference: where the relevant code lives

File	Function / Line	Purpose
<code>app/sw.js</code>	<code>fetch</code> listener (around L72–L104)	SW fetch handler, network-first
<code>app/sw.js</code>	<code>CACHE_VERSION</code> constant (L20)	Cache namespace; bump to force eviction
<code>app/app.js</code>	<code>serviceWorker.register</code> (L479)	SW registration call

Note

Line numbers are accurate as of 2026-06-11. They drift with edits — search by name (`CACHE_VERSION`, `serviceWorker.register`) if the lines no longer match.